École Polytechnique Fédérale de Lausanne

EPFL Spacecraft Team

Firmware Architecture and Verification of the Electrical Power System of the CHESS CubeSat

---

Walid Belmo – [SCIPER]

Semester Project

Supervisor: [Professor's NAME Surname]

May 15, 2026

Abstract

The CHESS (CubeSat for High-Energy Spectroscopy) mission, currently under development at the EPFL Spacecraft Team, requires an Electrical Power System (EPS) capable of managing solar-array energy capture, battery charging, load distribution, and autonomous safety behaviour. This report describes the design, implementation, and hardware-in-the-loop bench-test plan for the firmware that runs on the EPS microcontroller — a Microchip SAMD21 — and decides, in real time, how the satellite's electrical subsystem operates.

The firmware is bare-metal C running in a single super-loop without an operating system or vendor hardware abstraction layer. It implements a Power Conditioning Unit (PCU) state machine with four normal operating modes and a safe-mode override, an Incremental Conductance Maximum Power Point Tracking (MPPT) algorithm, a complementary 300 kHz PWM driver for the EPC2152 GaN half-bridge buck converter, the CHIPS framing protocol that the satellite uses for all internal serial communication, and a sensor abstraction that lets the same code consume either injected test values or real ADC and I^2C readings.

A hardware-in-the-loop test harness, built from a Python web application, an ESP32 serial bridge, and the real PCU testing board V4.1, is presented. Twelve preset scenarios exercise every base case of the state machine; an MPPT convergence page demonstrates the algorithm tracking a configurable solar-panel current–voltage curve in closed loop; a manual-control page lets an operator drive every board output independently. A nine-test bench-measurement program is described that produces the physical evidence required to support the claim that each firmware capability was verified on the real hardware.

The report closes by enumerating the differences between the present simulator-grade implementation and the flight-grade firmware the mission ultimately requires, and outlines a clearly-staged path between the two.

Contents

Acronyms and Abbreviations

ADC	Analog-to-Digital Converter
BOM	Bill Of Materials
CHESS	CubeSat for High-Energy Spectroscopy
CHIPS	CHESS Internal Protocol over Serial
CRC	Cyclic Redundancy Check
CV	Constant Voltage (charging phase)
DFP	Device Family Pack (Microchip)
DMA	Direct Memory Access
EPC2152	Efficient Power Conversion 2152 GaN half-bridge driver
EPFL	École Polytechnique Fédérale de Lausanne
EPS	Electrical Power System
FSM	Finite-State Machine
GaN	Gallium Nitride
GPIO	General-Purpose Input/Output
HAL	Hardware Abstraction Layer
HIL	Hardware-In-the-Loop
I^2C	Inter-Integrated Circuit (serial bus)
INA226	Texas Instruments INA226 current/voltage monitor
JPL	Jet Propulsion Laboratory
LEOP	Launch and Early Operations Phase
MCU	Microcontroller Unit
MPP	Maximum Power Point
MPPT	Maximum Power Point Tracking
OBC	On-Board Computer
PCB	Printed Circuit Board
PCU	Power Conditioning Unit
PDU	Power Distribution Unit

PGOOD	Power Good (status signal)
PWM	Pulse-Width Modulation
RTOS	Real-Time Operating System
SAMD21	Microchip SAM D21 microcontroller family
SERCOM	Serial Communication block (SAMD21 peripheral)
SoC	State of Charge
SPI	Serial Peripheral Interface
SWD	Serial Wire Debug
TCC	Timer/Counter for Control applications (SAMD21 peripheral)
TP	Test Point (on the PCB silkscreen)
TPS25940	Texas Instruments TPS25940 eFuse IC
UART	Universal Asynchronous Receiver/Transmitter
UVLO	Under-Voltage Lockout

1 Introduction

1.1 The CHESS mission

CHESS (CubeSat for High-Energy Spectroscopy) is a CubeSat mission in development at the EPFL Spacecraft Team. The satellite's payload is a gamma-ray spectrometer designed to observe high-energy astrophysical sources from low Earth orbit. CHESS is a student-led project: each subsystem — structure, attitude, thermal, communications, on-board computer, and electrical power — is owned by a team of student engineers, with regular reviews against a master mission document that defines requirements, modes, and interfaces.

The work described in this report concerns the *Electrical Power System* (EPS): the spacecraft subsystem that generates, stores, and distributes electrical energy. The EPS is one of the few subsystems that must operate before any other on the satellite, because every other subsystem — including the On-Board Computer (OBC) that nominally supervises the spacecraft — needs the EPS to provide power before it can boot.

1.2 The role of the EPS

The CHESS EPS is built on three functional blocks:

Power Conditioning Unit (PCU) Captures energy from the solar array, regulates the charging current into the battery, and protects the array–battery interface. The PCU contains the buck converter, the maximum-power-point-tracking algorithm, and the four-mode operating logic that governs how energy is converted in different spacecraft conditions.

Power Distribution Unit (PDU) Provides regulated 3.3 V, 5 V, and battery-bus rails to the rest of the satellite. The PDU contains eFuses that the EPS can enable or disable through digital pins, giving the satellite software-controlled load shedding for contingency operations.

EPS microcontroller (MCU) Reads sensors, runs the control and state-machine logic, commands the PCU and PDU actuators, reports telemetry to the OBC, and falls back to autonomous behaviour if the OBC stops responding. The MCU is a Microchip SAMD21 — a 32-bit Cortex-M0+ core with 128 kB flash, 16 kB RAM, and no external operating system. All firmware described in this report runs on this chip.

1.3 Scope of this semester project

The boundary of this semester project is the firmware that runs on the MCU. The PCB hardware was designed by an earlier generation of the team; component placement, schematic, and bill-of-materials are fixed and were available throughout the work.

Inside that boundary, the project covered:

- designing a clean, layered firmware architecture from the existing working but ad-hoc code base;
- implementing the PCU operating-mode state machine in pure C, without hardware dependencies, so that it could be tested on a laptop before any board was involved;
- implementing the Incremental Conductance MPPT algorithm with the same separation between pure logic and hardware;
- implementing the complementary 300 kHz PWM driver with hardware dead-time for the EPC2152 GaN half-bridge that drives the buck converter;
- implementing the CHIPS framing protocol used by every CHESS subsystem to talk to the OBC;
- building a hardware-in-the-loop demonstration harness centred on a local Python web application and an ESP32 serial bridge, so that the firmware running on the real chip can be commanded, inspected, and exercised end-to-end from a laptop;
- bringing up the real PCU testing board V4.1, including the external programmer (Atmel-ICE through a Tag-Connect cable), the buck-stage PWM scope verification, and a first round of injected sensor-value tests;
- planning, but not yet executing, a structured bench programme to physically verify every firmware capability against known electrical stimuli.

Out of scope for this semester were the physical sensor drivers for the digital current monitors (INA226) and the SPI battery-temperature sensor (whose part number is not yet specified in the mission document), and the eventual integration of every subsystem into a single super-loop running flight-grade per-mode logic. Both are described in the future-work chapter as the next clearly-defined steps.

1.4 What “done” means for this report

The semester deliverable is not a flight-ready EPS: the mission’s operational thresholds are not yet finalised, the battery part has not been selected, and the spacecraft thermal model is still in development. What the project does deliver is a firmware architecture that can already be exercised end-to-end against real hardware, plus the test infrastructure to validate every claim made about it.

Concretely, “done” for this report means:

1. every firmware capability listed in Section 1.3 is implemented and compiles cleanly under the JPL/MISRA-derived coding rules of the project;
2. the firmware has been flashed and exercised on the real PCU V4.1 mainboard, with the OBC link (an ESP32 acting as a stand-in) verified by valid CHIPS responses;
3. the four PCU operating modes and the safe-mode entry triggers have been exercised against injected sensor values, with each expected mode reached;

4. the MPPT algorithm has been exercised against a software solar-panel model on the ESP32, with closed-loop duty-cycle convergence observed;

5. a structured bench-test plan exists with primary-source probe-point references, so that any team member can take the project forward without re-discovering the same PCB facts.

1.5 Report structure

The remainder of the report is organised as follows. **Chapter 2** introduces the mission requirements, the electrical concepts (MPPT, buck converters, complementary PWM, eFuses) needed to follow the rest of the report, and the SAMD21 platform. **Chapter 3** describes the firmware architecture and walks through each major module. **Chapter 4** describes the hardware-in-the-loop test harness — the web app, the ESP32 bridge, and the demo pages — and the bench-test programme planned to physically verify each firmware capability. **Chapter 5** summarises the verified behaviour, lists the gaps between the simulator-grade implementation and a flight-grade EPS, and proposes a staged path between them.

2 Background and Requirements

This chapter introduces the electrical concepts a reader needs to follow the firmware design — solar-panel power transfer, MPPT, buck conversion, complementary PWM, eFuses, framed serial communication, and the SAMD21 platform — and the requirements inherited from the CHESS mission document.

2.1 The EPS in one diagram

The shortest useful model of the EPS is a chain:

`solar arrays + battery → PCU conditions and protects energy → PDU distributes protected rails → MCU measures, decides, commands, reports.`

The PCU is responsible for the energy-conversion physics. The PDU distributes the conditioned rails to the loads (OBC, payload, radio, attitude control, ADCS) through eFuses. The MCU — the subject of this report — is the supervisor that ties the two halves together and speaks to the OBC.

2.2 Operating modes of the PCU

The PCU has four normal operating modes, each driven by the current combination of solar availability and battery state of charge.

`MPPT_CHARGE` The Sun is available and the battery is not yet full. The PCU draws as much energy as possible from the panels and routes it to the battery. This is where the MPPT algorithm is active.

`CV_FLOAT` The battery is at or near its maximum safe voltage. The PCU stops trying to maximise panel power and instead regulates the charging rail to a constant voltage, letting charge taper off naturally.

`SA_LOAD_FOLLOW` The battery is fully charged and the Sun is still available. The PCU runs the array only hard enough to cover the spacecraft's instantaneous load, avoiding overcharge.

`BATTERY_DISCHARGE` No usable sunlight (the satellite is in eclipse, or the panels are still stowed). The panel rail is opened and the battery powers everything.

Above these four sits the `SAFE_MODE` state, an emergency override entered when the battery voltage drops below a critical threshold, the battery temperature is outside its safe operating range, or the OBC has been silent for longer than the heartbeat timeout. Safe mode has four sub-states selected by the OBC (`DETUMBLING`, `CHARGING`, `COMMUNICATION`, `REBOOT`); each sub-state defines the load mask that must stay powered while the satellite is in survival configuration.

The transition rules and the threshold definitions are documented in `state-transitions.md` inside the project repository, and implemented in pure C as a single dispatcher function described in Chapter 3.

2.3 Maximum Power Point Tracking

A solar panel does not behave as a constant voltage source. Its current–voltage (I–V) curve has a single point of maximum power output that depends on irradiance, temperature, and load. The job of an MPPT algorithm is to actively steer the converter's operating point onto that maximum.

The mission document selects Incremental Conductance as the MPPT algorithm of choice. Incremental Conductance compares the instantaneous conductance I/V to the incremental conductance $\mathrm{d}I/\mathrm{d}V$: at the maximum power point the sum of the two is zero. Adjusting the buck converter's duty cycle moves the operating point along the I–V curve, and the algorithm walks the duty cycle up or down to drive that sum toward zero. Compared to Perturb-and-Observe, Incremental Conductance is less prone to oscillation around the peak and tracks faster under changing irradiance.

2.4 The buck converter and complementary PWM

The PCU's energy-conversion element is a step-down (buck) DC–DC converter built around an EPC2152 GaN half-bridge. The EPC2152 needs two synchronised gate-drive inputs — high-side and low-side — that are complementary: when one is on, the other must be off. If both were on simultaneously, even for nanoseconds, the half-bridge would short the input rail (a condition called *shoot-through*) and the FETs would be destroyed.

The complementary drive is generated by the SAMD21's TCC0 timer peripheral. Both gates run at 300 kHz with the duty cycle controlled by software. A small *dead-time* interval, during which both gates are off, is inserted at every transition. The SAMD21 datasheet specifies the dead-time generator as a hardware feature of the TCC peripheral: the firmware programs two registers (`DTLS` and `DTHS`) to set the low-side and high-side dead times in units of the clock period.

For the CHESS hardware the dead time is set to approximately 100 ns. At 48 MHz core clock this corresponds to five clock periods. The value was confirmed visually on an oscilloscope during the mainboard bring-up.

2.5 The CHIPS protocol

CHIPS (CHESS Internal Protocol over Serial) is the framing protocol used across the satellite for every OBC–subsystem exchange. The mission document defines its frame structure:

- Two sync bytes (`0x7E`) delimit every frame.
- A one-byte sequence number, a one-byte command/response byte (with the high bit indicating direction), and a variable-length payload follow.

- A two-byte CRC-16/KERMIT checksum is appended before the trailing sync byte.
- Byte stuffing (PPP-style, RFC 1662) escapes any payload byte equal to `0x7E` or `0x7D`.

The OBC is always the master: it issues commands; each subsystem acknowledges within one second. If the OBC sends the same sequence number twice in a row (because its first request timed out), the subsystem must execute the command only once but reply each time the request arrives. After 120 seconds without an OBC message, a subsystem may assume autonomy.

2.6 The SAMD21 platform

The MCU is a Microchip SAMD21J17D-MUT on the real PCU board (64-pin QFN, 128 kB flash, 16 kB RAM) and a SAMD21G17D on the Curiosity Nano dev board used for early bring-up (48-pin TQFP, same silicon die). Both are Cortex-M0+ cores at up to 48 MHz. The choice of bare-metal C without HAL or RTOS is deliberate: the same flash budget that buys a HAL on a richer chip is needed here for the application logic itself, and direct register access makes real-time behaviour straightforward to audit.

The relevant peripherals used by the EPS firmware are TCC0 for the buck PWM, SERCOM0 for the OBC UART, SERCOM3 for the I^2C bus to the INA226 chips, and the ADC for the analog sensor inputs.

2.7 Mission requirements traceability

The detailed requirement set lives in the project repository as `docs/requirements.md`. The most relevant requirements for this report are reproduced in Table 1; each maps to a section of the architecture chapter or the test plan.

Every requirement is traceable to a section of the implementation (Chapter 3) and to a verification step (Chapter 4).

Table 1: Selected mission requirements relevant to this report.

ID	Requirement
PCU-1	The PCU shall use a buck converter driven by complementary PWM with dead-time.
PCU-2	The buck converter shall switch at 300 kHz.
PCU-3	The PCU shall run MPPT when solar power is available and the battery needs charge.
PCU-4 to PCU-7	The PCU shall support the four operating modes `MPPT_CHARGE`, `CV_FLOAT`, `SA_LOAD_FOLLOW`, `BATTERY_DISCHARGE`.
MCU-1	The MCU shall boot first and manage power before the OBC is available.
MCU-3 to MCU-5	The MCU shall read PCU/PDU electrical telemetry via I^2C and ADC.
MCU-9	The MCU shall communicate with the OBC over CHIPS.
MCU-12	The MCU shall enter autonomous behaviour after 120 s of OBC silence.
SAFE-1 to SAFE-8	The MCU shall report critical conditions, shed non-essential loads, activate heaters when needed, and not exit safe mode autonomously.

3 Firmware Architecture and Implementation

This chapter walks through the firmware running on the EPS microcontroller. It begins with the design philosophy that shapes every decision below, then describes each major module — the main loop, the state machine, the MPPT algorithm, the PWM driver, the sensor abstraction, the CHIPS codec, and the manual-control mode — and closes with the conventions that the code must comply with.

3.1 Design philosophy

The firmware is bare-metal C running in a single super-loop. There is no real-time operating system, no vendor hardware abstraction layer, no dynamic memory allocation after boot, and no recursion. Every loop or buffer has a compile-time-known upper bound. These constraints come directly from the NASA Power of 10 rules Holzmann 2006, the JPL Institutional Coding Standard for C Jet Propulsion Laboratory 2009, and MISRA C:2004 *MISRA C:2004 – Guidelines for the Use of the C Language in Critical Systems* 2004, and they exist for the same reason any flight-software project adopts them: behaviour that an auditor can reason about statically is much easier to argue safe than behaviour that depends on runtime memory pressure or scheduling order.

A second principle is the deliberate separation of *pure logic* from *hardware access*. Functions that compute — the state machine dispatcher, the MPPT iteration, the CHIPS frame codec, the CRC calculator — never touch registers and never use global state. They take input structs, return output structs, and can be compiled on a laptop and

tested in isolation. Functions that touch hardware — the SERCOM UART driver, the TCC PWM driver, the ADC reader, the I^2C master — never contain control logic. This split makes every algorithmic claim of this report testable without any chip on the bench.

3.2 Folder and naming conventions

The source tree is organised by responsibility, not by topic. The top-level project folder `src-pds/` contains one subfolder per architectural layer:

```
src-pds/
  app/                                main loop
  board_command_contract/             CHIPS command IDs and payloads
  board_hardware_startup/             clock, peripherals, GPIO init
  board_outputs/                      PWM and output-state writes
  communication_with_esp32/           CHIPS read/dispatch/reply
  externally_controlled_behaviors/    per-mode runners
  runtime_state/                      shared snapshot RAM
  sensor_inputs/                      layered sensor abstraction
  shared_helpers/                     byte-order helpers
  state_machine_pure_logic/           pure dispatcher function
  status_reporting_to_esp32/          telemetry builders
```

File names are sentences, not abbreviations. A reader inspecting the folder listing can tell what each file does without opening it: `functions_to_run_mppt_algorithm_with_selected_input_sou` `functions_to_apply_allowed_pwm_to_board_hardware.c`, `functions_to_initialize_board_hardwar` and so on. This style is taken directly from the project's `conventions.md` and is enforced on every file.

3.3 The main loop

The application entry point reads like a table of contents. Listing 1 reproduces the body of `main()` verbatim from `src-pds/app/main.c`, with only minor cosmetic shortening.

```
int main(void) {
    switch_mcu_clock_from_1mhz_to_48mhz();
    setup_sercom0_uart_registers_for_esp32_link();
    setup_millisecond_timer_for_loop_timing();
    setup_sensor_reading_hardware();
    setup_pwm_output_starting_off();
    setup_output_pins_starting_off();

    start_esp32_command_reader_with_empty_message_state();
    set_starting_runtime_state_to_safe_defaults();

    while (1) /* @non-terminating@ */ {
        read_and_execute_commands_from_esp32();
        toggle_status_led_for_heartbeat_if_due_and_not_in_manual_mode()
            ;

        if      (requested_mode_is_off())           run_off_mode();
        else if (requested_mode_is_flight())
            run_full_satellite_logic();
        else if (requested_mode_is_mppt_test())     run_mppt_test_only
            ();
```

```
19          else if (requested_mode_is_state_test())
               run_state_transition_test_only();
20          else if (requested_mode_is_fixed_pwm_test())
               run_fixed_pwm_test_only();
21          else if (requested_mode_is_manual())
               run_manual_control_mode();
22
23          block_dangerous_outputs();
24          apply_outputs_to_board();
25          send_status_if_needed();
26          reset_watchdog_timer();
27      }
28  }
```

Listing 1: The main super-loop of the EPS firmware.

The mode-dispatch block is one of the project's most explicit design choices. Rather than building a single monolithic "flight" loop that runs every behaviour simultaneously, the firmware exposes each capability as a separately commandable mode. Each mode runs independently, in isolation from the others. This keeps every individual capability testable; once each is verified, the integration into a single "flight" loop becomes one additional method, which is the `run_full_satellite_logic()` branch above.

3.4 The state machine

The PCU operating-mode state machine is implemented as a single pure function in `state_machine_pure_logic/functions_to_compute_next_pcu_state_and_actuator_commands_from_pure_logic.c`. The function signature takes a struct of sensor readings (battery voltage, panel voltage, panel current, temperature, OBC heartbeat, OBC-commanded mode, fault flags) and returns a struct of actuator commands (selected PCU mode, safe-mode sub-state, MPPT enable flag, heater enable flag, load mask, deploy-antenna flag, send-telemetry flag, reboot flag).

The dispatcher reduces to a short cascade of `if`/`else if` clauses: if any safety condition is tripped, return `SAFE_MODE`; otherwise return `BATTERY_DISCHARGE` (no sun), `MPPT_CHARGE` (below battery-full), `SA_LOAD_FOLLOW` (battery full), or `CV_FLOAT` (the intermediate band). The full flight FSM includes time-based transitions (the `t1` and `t2` debounce counters), hysteresis bands, and per-mode internal algorithms; these are explicitly listed as future work in Chapter 5.

A complementary verification function, the `dispatcher()`, is implemented separately and compared against the FSM output at steady state. This is the standard specification-versus-implementation cross-check that catches regressions when the FSM is extended.

3.5 The MPPT algorithm

The MPPT module (`src/mppt_algorithm.c`) implements Incremental Conductance in pure integer arithmetic. The algorithm reads the panel's raw ADC values for voltage and current, filters them through an 8-sample moving average, computes the IncCond decision, and returns a new duty cycle in the range $[0, 65535]$ that the PWM module will apply to the buck converter.

```c
uint16_t mppt_algorithm_run_one_iteration(
        struct mppt_algorithm_state *state,
        uint16_t raw_panel_voltage_adc,
        uint16_t raw_panel_current_adc) {

    update_moving_average_filter(state, raw_panel_voltage_adc,
       raw_panel_current_adc);

    int32_t delta_v = state->filtered_v - state->previous_v;
    int32_t delta_i = state->filtered_i - state->previous_i;

    /* Decide whether to step duty up or down based on di/dv
     * and i/v comparison; clamp to [DUTY_MIN, DUTY_MAX]. */
    state->duty_cycle = apply_incremental_conductance_decision(
            state, delta_v, delta_i);

    state->previous_v = state->filtered_v;
    state->previous_i = state->filtered_i;

    return state->duty_cycle;
}
```

Listing 2: Skeleton of the Incremental Conductance iteration.

Because the function takes only the algorithm's own state struct and the two ADC values, it can be run on a laptop with simulated ADC inputs. A reference simulator (a five-parameter single-diode panel model) was built for exactly this purpose during the dev-board phase of the project; the same algorithm now runs unchanged on the SAMD21.

3.6 The PWM driver

The PWM driver (`src/drivers/pwm_buck_converter_tcc0_pa12_pa13_on_mainboard.c`) configures TCC0 to produce two complementary outputs at 300 kHz on pins PA12 and PA13 with hardware-inserted dead-time. The driver exposes one initialisation function and one runtime function:

```c
void pwm_tcc0_initialize_complementary_300khz(void);
void pwm_tcc0_set_duty_cycle(uint16_t duty_as_fraction_of_65535);
```

The duty argument matches the MPPT output range, so the two modules plug together without any rescaling. Internally, the driver maps the 16-bit duty value to the TCC0 `CC` register range (0–160 for 300 kHz on the 48 MHz GCLK), and writes both compare channels under the dual-channel TCC0 dead-time scheme required by the actual PA12/PA13 routing on the mainboard.

The driver also implements an explicit *arm/disarm* safety gate. PWM output is disarmed on boot, on every mode change, and on any injected-fault flag. Re-arming requires an explicit `pwm-arm` command from the host. This prevents the buck stage from being unintentionally energised during firmware development.

3.7 The sensor abstraction

The firmware reads sensor values through a three-layer abstraction (Figure 1). The state machine and the MPPT algorithm at the top call exactly five functions — `read_battery_voltage()`, `read_battery_current()`, `read_panel_voltage()`, `read_panel_current()`, `read_charging_rail_volt` — and get back one number each in engineering units. They never touch the ADC, I^2C, or any specific chip. A single runtime flag selects whether those numbers come from injected operator values (the default during development) or from real PCB hardware. The hardware path itself is split per chip (INA226, LT6108, TPS25940 IMON, resistor dividers), each behind its own small driver.

[Figure placeholder]
Sensor abstraction in three layers: top layer = five `read_X()` functions called by state machine and MPPT; middle layer = per-chip drivers (INA226, LT6108, TPS25940 IMON, voltage dividers) plus the injected-value branch; bottom layer = two bus drivers (ADC reader and I^2C master).

Figure 1: Three-layer sensor abstraction. The top layer hides the sensor-source decision from the rest of the firmware.

This split is what lets the same firmware run on the dev board (no real sensors — everything falls back to the injected value) and on the real mainboard (the same hardware path now drives actual silicon).

3.8 The CHIPS frame codec

The CHIPS protocol is implemented as a pure-logic codec in `chips_protocol_encode_decode_frames_w` The encoder builds a framed byte stream from a (sequence, command, payload) tuple; the decoder is a byte-fed state machine that accumulates bytes between sync markers, runs the CRC, and emits either `CHIPS_FRAME_READY`, `CHIPS_INCOMPLETE`, or `CHIPS_ERROR`. Byte stuffing is applied between the two sync markers, exactly as specified by the mission document.

On the SAMD21 side, the codec is wrapped by a thin transport layer that reads bytes from the SERCOM0 UART (interrupt-driven, with 256-byte ring buffers in both directions). On the ESP32 side, the same codec runs in an Arduino sketch so that the test harness can build valid frames from the laptop's plain-text commands.

3.9 Manual control mode

A separate operating mode lets an operator drive every user-facing board output independently from a web page, without involving the state machine or the MPPT loop. The mode is selected by the `enter_manual` CHIPS command; four subsequent commands (`set_manual_pwm`, `set_manual_pv`, `set_manual_bat`, `set_manual_led`) write directly to the corresponding actuator. This mode exists primarily for hardware bring-up and one-off bench verifications; it is not part of any flight scenario.

3.10 Coding conventions

Every file in the project is required to comply with the rules documented in `conventions.md`. The most-binding rules include: names are sentences not abbreviations; functions do one job; assertions stay active in flight builds; zero compiler warnings under `-Wall -Wextra -Werror -pedantic`; fixed-width integer types only; `volatile` on every variable shared between an ISR and the main loop; one statement per line; explicit parentheses in every compound expression. Each of these rules has a stated rationale, and most are direct translations of either the Power of 10 list Holzmann 2006 or the JPL standard Jet Propulsion Laboratory 2009.

The total project compiles cleanly under `-Wall -Wextra -Werror` for both the `BOARD=devboard-pds` and `BOARD=mainboard-pds` targets.

4 Test Harness and Hardware-in-the-Loop Demonstration

This chapter describes the hardware-in-the-loop (HIL) test infrastructure built around the firmware: the multi-tier architecture (web app, ESP32 bridge, SAMD21 board), the demo pages exposed to the operator, the twelve preset scenarios used to exercise the state machine, the MPPT convergence demo, the real-mainboard bring-up, and the bench-test programme that produces the physical-measurement evidence backing each firmware claim.

4.1 Why hardware-in-the-loop

A pure-software simulator can prove that an algorithm produces the right answer for a clean set of inputs. It cannot prove that the algorithm produces the right answer *on the actual chip*, with real ADC noise, real interrupt latency, real peripheral configuration, and real wiring. Conversely, a pure hardware test — a board with a solar panel, a battery, and a load — is expensive to set up, hard to script, and gives slow feedback when an algorithm needs adjustment.

The HIL approach takes a middle path. The firmware runs unchanged on the real chip on the real board. The sensor environment — the solar panel and the battery — is replaced by a software model running on an external companion microcontroller. The companion exchanges raw ADC values with the SAMD21 over a serial link, closing the loop in real time. Anything the firmware does on real hardware (peripheral configuration, interrupt handling, timing) is exercised. Anything the firmware does with the environment (algorithm decisions, state transitions) is exercised against scriptable, repeatable stimuli.

4.2 The architecture of the harness

The harness has three tiers:

The laptop. A Python web application serves three local pages (`/mppt`, `/state`, `/manual`) at `http://127.0.0.1:8000`. Each page renders an HTML/CSS/JS front-end and posts text commands to a small set of `/api/...` endpoints. The application owns a single USB-serial connection.

The ESP32 bridge. An Arduino sketch on an ESP32 dev-board acts as the OBC stand-in. It translates the plain-text commands forwarded from the laptop into binary CHIPS frames, sends them to the SAMD21 over a 3.3 V UART link, parses the replies, and prints them back to the laptop in a regex-friendly format.

The SAMD21. The real EPS firmware. It receives CHIPS frames on SERCOM0, dispatches commands, runs the selected mode, applies outputs, and streams status replies back to the bridge.

[Figure placeholder]
Three-tier HIL architecture: browser → Python web app (`run_local_web_app.py`) → USB serial (`COM3`) → ESP32 bridge → 3.3 V UART → SAMD21 mainboard. CHIPS frames flow in both directions on the 3.3 V link; plain text on the USB link.

Figure 2: The three-tier hardware-in-the-loop architecture.

The advantage of routing everything through one port and one Python process is that no second connection has to be opened when a new page is added: a new page becomes a new actions module and a new `static/<page>/` folder; the existing serial link is reused. The HTML pages share a single live-telemetry sidebar that updates from the streamed snapshots.

4.3 The three demo pages

4.3.1 MPPT convergence

The `/mppt` page lets the operator configure a simulated solar-panel I–V curve (open-circuit voltage, short-circuit current, voltage at the maximum power point), launch the MPPT demo, and watch the algorithm converge in real time. The ESP32 runs the physics — the simulated panel and the buck converter model — and the SAMD21 runs the algorithm. The page plots panel voltage, panel current, panel power, and commanded duty cycle on a shared time axis, marking the theoretical maximum- power point as a horizontal line. Convergence is visually obvious to a reviewer who is not familiar with the algorithm.

4.3.2 State-transition scenarios

The `/state` page exposes twelve preset scenarios that each inject a steady set of sensor values and assert which state the firmware should reach. The scenarios cover the four normal PCU modes (`MPPT_CHARGE`, `CV_FLOAT`, `SA_LOAD_FOLLOW`, `BATTERY_DISCHARGE`), the three safe-mode entry triggers (battery low, temperature out of range, heartbeat lost), and the four safe-mode sub-state load masks. Running a scenario sends the sensor values, waits a few seconds for steady state, reads back the firmware's snapshot, and compares the observed mode and output mask to the expected values. A green dot means the scenario passed; a red dot opens a side-by-side table of expected versus observed values.

A run-all button executes the suite in sequence and reports a single pass/fail count. The current implementation reaches 12 of 12 passes against the pure-dispatcher firmware running on the real board.

4.3.3 Manual control

The `/manual` page lets the operator drive each of the four user-facing board outputs independently: the PWM duty cycle, the photovoltaic (PV) eFuse enable, the battery (BAT) eFuse enable, and the green status LED. The page enforces an "Off first" convention before allowing entry into manual mode and disarms the PWM on every disconnect.

Manual mode is primarily used during hardware bring-up when a specific output needs to be observed in isolation.

A fourth page, `/comm` (communication and status), is listed in the project plan but is not yet implemented.

4.4 Real-mainboard bring-up

The CHESS EPS firmware was first developed against the Curiosity Nano DM320119 dev board (SAMD21G17D) and then ported to the real PCU testing board V4.1 (SAMD21J17D-MUT). The two chips are the same silicon die in different packages, so the register-level code is unchanged; only the pin assignments, the chip define, the startup file, and the linker script differ between the two builds.

The mainboard bring-up established:

- the programming path: Atmel-ICE through a Tag-Connect TC2050-IDC-NL footprint at J1, with SWD clock at 125 kHz;
- the programming software path: Microchip Studio Device Programming (the custom-cable adapter does not connect reset, so reset-dependent flows could not be used);
- the CHIPS link: the ESP32 bridge wired to J3 (`TX` to PA11, `RX` to PA10, common GND) exchanges valid frames at 115200 baud;
- the full firmware (`satellite_firmware.hex`) runs on the real chip; telemetry, state-machine scenarios, MPPT demo, and PWM safety gating all behave as on the dev board;
- the buck PWM signals on PA12/PA13 were observed on an oscilloscope, with a measured dead-time gap of approximately 100 ns at the first checked edge.

The second PWM edge and a full duty-cycle sweep are part of the planned bench programme (Section 4.4.5).

4.5 The bench-test programme

The HIL harness exercises algorithm logic. To complete the verification chain the firmware also needs to be exercised against *physical electrical stimuli*: known voltages applied to ADC inputs, known currents through the shunt resistors, a real buck converter producing real V_{out}, and a real fake-solar source whose maximum power point can be computed by hand.

A structured nine-test programme has been planned and is documented in detail in `src-pds/how_to_test.md`. Table 2 summarises each test, its dependencies, and the evidence it produces for the report.

The plan also captures the dependencies between the tests (Test G requires the I^2C address fix; Test E requires the fake-solar power resistors; Tests B, D and the MPPT-on-real- sensors workflow require a host-reachable `set_sensor_source` command). Each test

Table 2: The nine bench tests and what each produces.

ID	Test	Evidence produced
A	PWM waveform sign-off	Scope shots of both edges at three duty cycles; dead-time table; frequency measurement.
B	ADC calibration vs known voltage	Calibration table per sensor net; observed vs reference ADC reference voltage.
C	eFuse switching on real board	Multimeter pre/post readings; PGOOD readback table; verified GPIO actuation.
D	State machine on real sensors	Mode-transition table with hardware-fed inputs; cross-check against the 12-scenario suite.
E	MPPT against a fake solar source ($\star$ headline)	Convergence plot from a known Thevenin source; comparison to the hand-calculated maximum power point.
F	Buck transfer curve	Duty vs $V_{\text{out}}/V_{\text{in}}$ plot showing the expected straight-line relationship.
G	INA226 sensor verification	Bus-scan output; ID register reads; multimeter cross-check at two current points.
H	CHIPS communication stress	Total-frames count after a ten-minute stream; bad-CRC and duplicate-sequence behaviour.
I	Watchdog timer	Uptime trace around a deliberate stall; verification of reset and recovery.

prescribes the exact silkscreen designators, test points, and PCB coordinates for every probe location, derived from a primary-source read of the public PCB schematic and BOM (Chapter 5 discusses the research process). The user running the test does not need to read the schematic itself.

When the bench programme is executed, the resulting tables, plots, and scope shots will populate the figures in Chapter 5.

5 Results, Discussion, and Future Work

This chapter summarises what is verified, what is not yet verified but planned, what the firmware deliberately does not do, and the staged path between the present implementation and a flight-ready EPS.

5.1 What is verified

The following items are verified to be working on the real PCU testing board V4.1 at the time of writing:

- The board boots, the 48 MHz clock initialises, the status LED blinks, and the watchdog is fed by the main loop.
- The CHIPS link on J3 exchanges frames bi-directionally with the ESP32 bridge at 115200 baud. The bus scan, the `telemetry`, the `state`, the `adc`, the injected-sensor commands, and the PWM safety commands all return valid responses.
- The twelve state-transition scenarios pass against the pure-dispatcher firmware (12/12 green).
- The MPPT demo runs in closed loop against the ESP32-side physics model. Convergence is visually clear on the web page; the duty cycle settles within tens of iterations.
- The complementary PWM at 300 kHz on PA12/PA13 has been scoped at one edge, with a dead-time gap of approximately 100 ns. The PWM safety gate disarms on boot, on mode change, and on any injected fault.
- Manual mode toggles each of the four user-facing outputs on the dev board, with the snapshot mirror behaviour matching the requested values even on outputs that are not physically wired on the dev board.

5.2 What the bench programme will verify

The nine-test bench programme described in Chapter 4.4.5 produces the remaining evidence. Once executed, the following statements will be supported:

- The PWM dead-time is consistent across the duty range and on both edges. (Test A.)
- Every ADC sensor channel reads correctly against a known applied voltage, with calibration coefficients quoted. (Test B.)
- Each eFuse can be enabled and disabled by firmware commands; the post-eFuse rail and the PGOOD telemetry track the commanded state. (Test C.)

- The state machine reaches the expected mode for each sensor combination when fed by real ADC instead of injected values. (Test D.)
- The MPPT algorithm running on the real chip converges to within a small percentage of the hand-calculated maximum power point of a fake-solar source (bench supply with a series resistor). (Test E — the headline figure of the report.)
- The buck stage follows the ideal step-down relationship across the operating duty range. (Test F.)
- The two INA226 chips respond on the I^2C bus at their schematic-derived addresses and produce readings that agree with a multimeter. (Test G.)
- The CHIPS link tolerates sustained streaming traffic without data loss and rejects corrupted frames as required by the protocol. (Test H.)
- The watchdog fires on a deliberate stall and the board recovers cleanly. (Test I.)

A blank table that the user will fill in as each test is executed is included in the appendix.

5.3 What the simulator deliberately does not prove

The state machine running today is a single short pure function that maps sensor values to a PCU mode. It is the simplest implementation that satisfies the steady-state specification, and it deliberately omits the per-mode machinery that flight-grade firmware will need. The `state-transitions.md` file inside the repository enumerates these omissions; the most important are:

- Timer-based transitions. The flight FSM moves from `MPPT_CHARGE` to `CV_FLOAT` only after the battery has been at full voltage for at least `t1` consecutive cycles. The current dispatcher detects "battery full" instantaneously.
- The CV_FLOAT `TEMP_MPPT` sub-state. A transient large load that briefly drains the battery while CV_FLOAT is active should trigger a temporary return to the MPPT algorithm. The current dispatcher does not implement this.
- Per-mode duty algorithms. Each mode in flight uses its own control law for the buck duty cycle (incremental conductance in `MPPT_CHARGE`, bang-bang voltage regulation in `CV_FLOAT`, current-clamped operation in `SA_LOAD_FOLLOW`, minimum-duty in `BATTERY_DISCHARGE`). The simulator uses placeholder duty values for everything but `MPPT_CHARGE`.
- Hysteresis and noise robustness. Real sensor readings jitter. The flight FSM debounces threshold crossings; the simulator receives clean injected values.
- Load shedding inside `BATTERY_DISCHARGE`. When the discharge current exceeds the safe limit the flight FSM is supposed to shed loads one at a time in priority order. The simulator does not implement this.
- Fault-flag interpretation. The ESP32 bridge can inject a fault bitmask; the current dispatcher does not yet decode any bit of it.

- Tuned threshold values. Every numeric threshold (battery maximum voltage, battery minimum voltage, temperature limits, heartbeat timeout) is a placeholder. Real values depend on the chosen battery cell, the thermal model, and the orbit profile, and will be set when the relevant hardware decisions are made.

These omissions are not bugs: they are explicit scope choices that keep the simulator-grade firmware small enough to be inspected and verified end-to-end during the project's duration.

5.4 Path to flight code

The work remaining to grow the present implementation into a flight-grade EPS is itemised in the `what_the_simulator_does_and_what_real_flight_software_still_needs.md` file in the repository. The eleven steps are summarised here:

1. Add a control-loop iteration counter to the firmware's shared state. This is the foundation for every timing-based behaviour.
2. Implement the `t1` timeout for the `MPPT_CHARGE` → `CV_FLOAT` transition.
3. Implement the `t2` debounce for the return transition.
4. Add the `TEMP_MPPT` sub-state inside `CV_FLOAT`.
5. Replace the placeholder duty cycle in `MPPT_CHARGE` with the real Incremental Conductance algorithm. (Already implemented in pure C; only the integration is pending.)
6. Implement bang-bang voltage regulation inside `CV_FLOAT`.
7. Implement the current-clamp duty algorithm inside `SA_LOAD_FOLLOW`.
8. Implement per-iteration load shedding inside `BATTERY_DISCHARGE`.
9. Add low-pass filters and hysteresis bands for every sensor channel.
10. Define and interpret the fault-flag bits.
11. Replace every placeholder numeric threshold with the mission-tuned values once the corresponding hardware is finalised.

Each of these can be implemented and verified by adding a new scenario to the existing twelve-scenario web page; the test infrastructure does not need to change.

5.5 Open issues

A small set of mission-level open questions affect the firmware but cannot be answered inside the firmware itself:

- The battery cell has not been chosen, so the values of $V_{\text{bat,max}}$, $V_{\text{bat,full}}$, $V_{\text{bat,min}}$, $V_{\text{bat,critical}}$, the maximum charge and discharge currents, and the temperature limits are all placeholders.
- The SPI-attached battery-temperature sensor is named in the mission document but the exact part is not specified, so its driver is currently a stub.
- The mission document and the team's communication choices disagree on whether the OBC–EPS link is I^2C or UART; the current firmware uses UART per the most recent team decision.
- The PCU V4.1 PWM routing requires the dual-channel TCC0 dead-time workaround documented in `pcb_design_error_for_pwm.md`; this is implemented in firmware but will need to be revisited when the next PCB revision is laid out.
- The PCB design contains a small documentation discrepancy: the EPC2152 reference designator on the silkscreen is `IC6`, whereas an older revision of the mainboard pinout document refers to it as `U1`. The firmware itself is unaffected; the documentation has been updated.

5.6 Conclusion

The work presented in this report delivers a firmware architecture for the CHESS EPS that is testable end-to-end on real hardware, decoupled by design between pure logic and hardware access, and structured so that the path from the present simulator-grade implementation to a flight-grade EPS is a sequence of well-scoped additions rather than a rewrite. The hardware-in-the-loop test harness, the twelve state scenarios, the MPPT convergence demonstration, and the nine-test bench programme together provide a level of verification coverage that is unusual for a one-semester student project, and that gives the next student to inherit the work a clean starting point.

The next student should source the power resistors required for the bench programme, execute the nine tests, and then begin the eleven-step climb to flight code. Each step is a single isolated change with a single new test scenario, and the architecture is ready to absorb them.

References

Holzmann, Gerard J. (2006). "The Power of 10: Rules for Developing Safety-Critical Code". In: *IEEE Computer* 39.6, pp. 95–97. DOI: 10.1109/MC.2006.212.

Jet Propulsion Laboratory (2009). *JPL Institutional Coding Standard for the C Programming Language*. Technical Report D-60411. NASA Jet Propulsion Laboratory.

MISRA C:2004 – Guidelines for the Use of the C Language in Critical Systems (2004). Motor Industry Software Reliability Association. Nuneaton, UK.

6 Appendix

6.1 Bench-test results template

The following table is provided blank; it is filled in as each bench test from Chapter 4 is executed. The date and the operator initials are recorded together with each pass/fail mark so that the same table also serves as the verification log for the report.

Table 3: Bench-test results log (to be filled in).

ID	Test	Date	Operator	Result	Notes
A	PWM waveform sign-off				
B	ADC calibration vs known voltage				
C	eFuse switching on real board				
D	State machine on real sensor source				
E	MPPT against a fake solar source				
F	Buck transfer curve				
G	INA226 sensor chip verification				
H	CHIPS communication stress				
I	Watchdog timer				

6.2 Build commands and repository layout

The firmware is built with the GNU ARM toolchain (`arm-none-eabi-gcc` 12.2.1) and flashed with OpenOCD or Microchip Studio. The two relevant build targets are:

```
make BOARD=devboard-pds   flash   # Curiosity Nano dev board
make BOARD=mainboard-pds  flash   # Real PCU testing board V4.1
```

The repository layout is summarised in Section 3.2. Every file inside `src-pds/` is described by its own file-naming convention; every file inside `src/drivers/` has a header comment indicating its build target (`devboard only`, `mainboard only`, or `shared`).

6.3 External references

The most useful external references for the firmware are listed below.

- Microchip SAMD21 datasheet (full chip and peripheral reference).
- Microchip ATSAMD21 Device Family Pack (DFP), v3.6.144 (register definitions and startup code used by the firmware).
- EPC2152 datasheet (the GaN half-bridge driving the buck).
- Texas Instruments INA226 datasheet (the digital current/voltage monitors on the I^2C bus).

- Texas Instruments TPS25940 datasheet (the eFuses on the panel and battery rails).
- NASA Power of 10 Holzmann 2006 and JPL Institutional Coding Standard for C Jet Propulsion Laboratory 2009 — the rule sources for the project's coding conventions.